

Experiment No.	9
Experiment Title	Perceptron vs Multilayer Perceptron (A/B Experiment) with Hyperparameter Tuning ¹
Student Name	Sanghamithra Namboodiri
Register Number	3122247001055
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	20th September 2026

1 Objective

To implement, tune and compare two neural classifiers on a 62-class handwritten character recognition task:

- **Model A:** a single-layer Perceptron Learning Algorithm (PLA) with a step activation, implemented from scratch using NumPy.
- **Model B:** a Multilayer Perceptron (MLP) with hidden layers and non-linear activations, whose activation function, optimizer, learning rate, depth/width and batch size are selected through systematic hyperparameter tuning on a validation set.

The two models are compared using accuracy, precision, recall, F1-score, confusion matrices, ROC curves (micro/macro average) and training-error convergence curves.

2 Problem Statement

Input: a grayscale image of a single handwritten character, resized to 32×32 pixels and flattened into a 1024-dimensional vector with values in $[0, 1]$.

Output: one of 62 class labels (digits 0--9, upper-case A--Z, lower-case a--z).

Objective: learn a mapping $f : \mathbb{R}^{1024} \rightarrow \{1, \dots, 62\}$ with a linear single-layer perceptron and with a tuned MLP, and quantify how much the non-linear, multi-layer model improves over the linear baseline.

3 Dataset Description

Dataset Name	English Handwritten Characters Dataset
Dataset Source	Kaggle (dhruvildave/english-handwritten-characters-da)
Number of Samples	3,410 images (all image paths in <code>english.csv</code> were resolved)
Number of Features	1,024 (32×32 grayscale pixels, flattened)
Number of Classes	62 (0-9, A-Z, a-z), about 55 images per class
Missing Values	None (no image paths were skipped while loading)
Train-Validation-Test Split	2,182 / 546 / 682 samples (64% / 16% / 20%), stratified, <code>random_state = 42</code>

¹GitHub Repository: <https://github.com/Sanghamithra259/ml-assignments>

4 Exploratory Data Analysis

- **Dataset overview:** `english.csv` maps each image path (e.g. `Img/img001-001.png`) to its character label. Every image was converted to grayscale, resized to 32×32 and flattened.
- **Statistical summary:** after division by 255 all pixel intensities lie in $[0, 1]$; the feature matrix has shape 3410×1024 .
- **Missing value analysis:** no missing labels or unreadable image files were encountered; the loader reported no skipped paths.
- **Class distribution:** the 62 classes are (near-)uniformly populated with about 55 images each, so the stratified test set contains 11 images per class.
- **Correlation matrix / feature distribution:** a 1024×1024 pixel correlation matrix is not informative for raw images (neighbouring pixels are trivially correlated), so class-balance and sample-image plots are used instead.

5 Data Preprocessing

- **Missing value handling:** not required; image paths that do not exist would be skipped and counted (none were).
- **Encoding:** string labels are mapped to integer indices $0, \dots, 61$ using `LabelEncoder`.
- **Feature scaling:** images are converted to grayscale ("L" mode), resized to 32×32 , flattened and normalised by dividing by 255 so that every feature lies in $[0, 1]$.
- **Train-validation-test split:** a stratified 80/20 split separates the test set; the remaining 80% is split again 80/20 into training and validation sets, giving 2,182 / 546 / 682 samples. The validation set is used only for hyperparameter selection, and the test set is used only for the final comparison.
- **Feature engineering:** none; models operate directly on raw pixel intensities.

6 Machine Learning Algorithm

6.1 Model A: Single-Layer Perceptron (PLA)

Working principle. The perceptron computes a weighted sum of the inputs and applies a step activation. Weights change only when a sample is misclassified. For the binary case

$$\hat{y} = \begin{cases} 1 & \mathbf{w}^\top \mathbf{x} + b \geq 0 \\ 0 & \text{otherwise} \end{cases}, \quad \mathbf{w}_{t+1} = \mathbf{w}_t + \eta (y - \hat{y}) \mathbf{x}. \quad (1)$$

For $K = 62$ classes, one weight vector $\mathbf{w}_k$ (with the bias appended) is kept per class and the prediction is $\hat{y} = \arg \max_k \mathbf{w}_k^\top \mathbf{x}$. On a mistake only the true-class and predicted-class rows are updated:

$$\mathbf{w}_y \leftarrow \mathbf{w}_y + \eta \mathbf{x}, \quad \mathbf{w}_{\hat{y}} \leftarrow \mathbf{w}_{\hat{y}} - \eta \mathbf{x}, \quad (2)$$

which reduces to the textbook rule above when $K = 2$. Samples are shuffled every epoch.

Advantages. Extremely simple and cheap; online, mistake-driven updates; converges in a finite number of updates if the data are linearly separable.

Limitations. It can only form linear decision boundaries; on non-separable data the weights keep oscillating and never converge; the step activation gives no probabilities and no gradient, so it cannot be trained with cross-entropy or backpropagation.

6.2 Model B: Multilayer Perceptron (MLP)

Working principle. Input $\rightarrow$ hidden layer(s) $\rightarrow$ softmax output. For a hidden layer l ,

$$\mathbf{h}^{(l)} = \phi(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}), \quad \phi \in \{\text{ReLU}, \tanh\}, \quad (3)$$

and the output layer produces class probabilities through the softmax function $p_k = e^{z_k} / \sum_j e^{z_j}$. Training minimises the multi-class cross-entropy loss (with a small L_2 penalty $\alpha = 10^{-4}$, the scikit-learn default)

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log p_{ik} + \frac{\alpha}{2N} \sum_l \|\mathbf{W}^{(l)}\|_F^2, \quad (4)$$

with gradients computed by backpropagation. Two optimizers were compared:

- **SGD** (mini-batch, with momentum 0.9 / Nesterov, scikit-learn defaults): $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$ plus momentum.
- **Adam**: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$, $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$, $\theta \leftarrow \theta - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$.

Advantages. Learns non-linear decision boundaries; produces class probabilities; hidden layers learn intermediate feature representations.

Limitations. Many hyperparameters; sensitive to learning rate, optimizer and input scaling; prone to overfitting when parameters greatly outnumber samples (the largest network here has about 3×10^5 weights for 2,182 training images); non-convex loss, so results depend on initialisation.

7 Experimental Setup

Python Version	3.12
IDE	VS Code
Operating System	Linux
Libraries	NumPy, pandas, scikit-learn, Pillow, Matplotlib, Seaborn

8 Hyperparameter Selection

8.1 PLA hyperparameters

Parameter	Value
Activation	Step / arg-max over class scores
Learning rate η	0.01
Epochs	50
Weight initialisation	$\mathcal{N}(0, 0.01^2)$, bias appended
Shuffling	Every epoch

8.2 MLP search space (grid search, 24 configurations)

Parameter	Values searched
Hidden layer sizes	(128), (128, 64), (256, 128)
Activation	ReLU, tanh
Optimizer (solver)	Adam, SGD
Initial learning rate	0.001, 0.01
Batch size	32
Loss function (fixed)	Cross-entropy (softmax output)
Maximum iterations (epochs)	150
L_2 penalty α	10^{-4} (default)
Early stopping	Off
Selection criterion	Validation accuracy

8.3 Tuning results

Table 2: Validation accuracy for all 24 MLP configurations (batch size 32, 150 iterations). The selected configuration is in bold.

#	Hidden layers	Activation	Solver	LR	Val. accuracy
1	(128)	ReLU	Adam	0.001	0.0147
2	(128)	ReLU	Adam	0.01	0.0147
3	(128)	ReLU	SGD	0.001	0.3791
4	(128)	ReLU	SGD	0.01	0.0147
5	(128)	tanh	Adam	0.001	0.0971
6	(128)	tanh	Adam	0.01	0.0165
7	(128)	tanh	SGD	0.001	0.3883
8	(128)	tanh	SGD	0.01	0.2527
9	(128, 64)	ReLU	Adam	0.001	0.0147
10	(128, 64)	ReLU	Adam	0.01	0.0147
11	(128, 64)	ReLU	SGD	0.001	0.4029
12	(128, 64)	ReLU	SGD	0.01	0.1905
13	(128, 64)	tanh	Adam	0.001	0.0403
14	(128, 64)	tanh	Adam	0.01	0.0165
15	(128, 64)	tanh	SGD	0.001	0.3938
16	(128, 64)	tanh	SGD	0.01	0.2930
17	(256, 128)	ReLU	Adam	0.001	0.0147
18	(256, 128)	ReLU	Adam	0.01	0.0147
19	(256, 128)	ReLU	SGD	0.001	0.4066
20	(256, 128)	ReLU	SGD	0.01	0.3846
21	(256, 128)	tanh	Adam	0.001	0.0495
22	(256, 128)	tanh	Adam	0.01	0.0165
23	(256, 128)	tanh	SGD	0.001	0.4139
24	(256, 128)	tanh	SGD	0.01	0.3718

8.4 Justification of the chosen configuration

The selected model uses **two hidden layers (256, 128)**, **tanh activation**, **SGD optimizer**, **learning rate 0.001** and **batch size 32**, cross-entropy loss (validation accuracy 0.4139).

- **SGD**, $\eta = 0.001$: for every architecture and activation, SGD with $\eta = 0.001$ outperformed SGD with $\eta = 0.01$ (all six pairs in Table 2); the smaller step gave smoother and more stable descent.
- **Adam rejected**: all Adam runs reached at most 9.7% validation accuracy and most stayed at $\approx 1.5\%$, which is chance level for 62 classes ($1/62 \approx 1.6\%$).
- **(256, 128)**: the wider two-layer network gave the best SGD results for both activations, although the gain over shallower networks is small (see Discussion).
- **tanh**: tanh and ReLU were within about one percentage point of each other under SGD; tanh was marginally better for the best (256, 128) network and is therefore selected.
- **Batch size 32**: kept fixed, since small mini-batches give many updates per epoch on a small (2,182-sample) training set.

9 Results

Metric (test set, macro average)	PLA (Model A)	MLP tuned (Model B)
Accuracy	0.1364	0.3915
Precision	0.2192	0.4405
Recall	0.1364	0.3915
F1-score	0.1212	0.3914
ROC-AUC (micro)	0.799	0.913
ROC-AUC (macro)	0.846	0.914

Table 3: A/B comparison on the 682-image test set. Macro recall equals accuracy because the stratified test set contains exactly 11 images per class.

Final MLP hyperparameters: hidden layers = (256,128), activation = tanh, solver = SGD, learning rate = 0.001, batch size = 32, 150 iterations.

10 Result Visualization

10.1 Model A: PLA

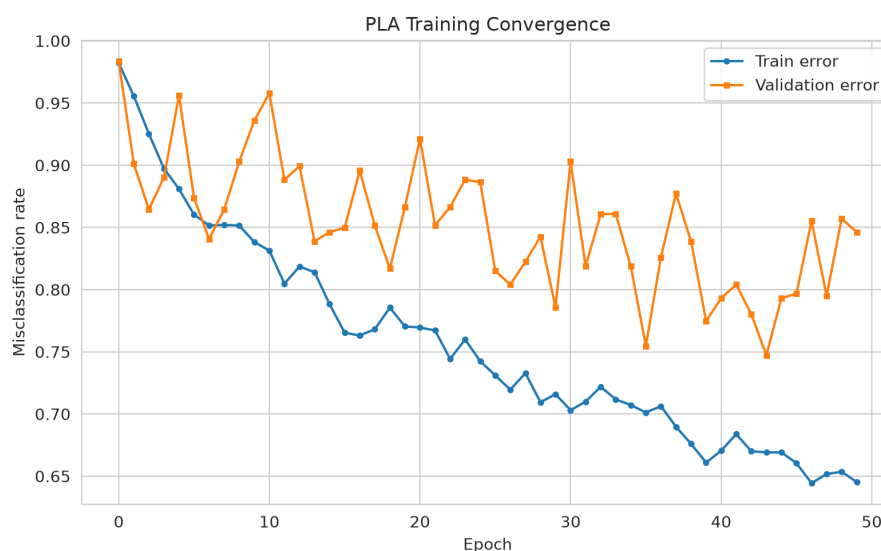


Figure 1: PLA training convergence: training and validation misclassification rate versus epoch.

Inference: The training error (fraction of mistakes made during each epoch) falls from 0.982 to 0.645 over 50 epochs, so the perceptron is learning something. The validation error, however, is noisy and does not follow it, jumping between about 0.75 and 0.96 (0.846 at epoch 50). This oscillation is the typical signature of the perceptron rule on non-linearly-separable data: each mistake pushes the weights, and the boundary never settles. The training curve is also an optimistic online estimate, since it is measured while the weights are still changing.

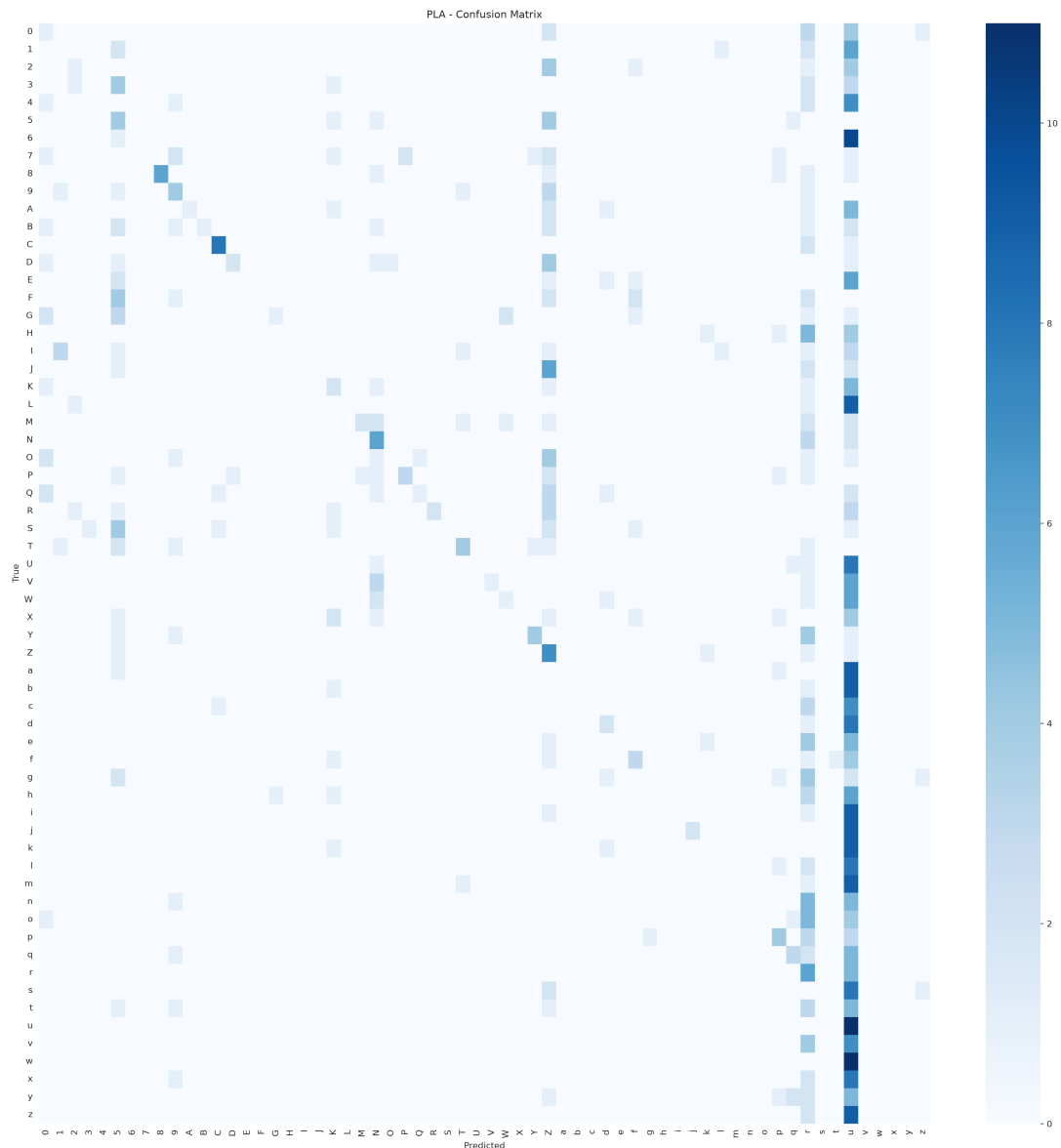


Figure 2: Confusion matrix of the PLA on the test set (62 classes).

Inference: With only 13.6% test accuracy, the diagonal of the PLA confusion matrix is weak and errors are scattered across many off-diagonal cells rather than concentrated in a few pairs. Predictions tend to collapse onto a subset of classes, which is consistent with macro precision (0.219) being higher than macro recall (0.136). Concretely, the matrix shows two dominant “sink” columns: lower-case u attracts stray predictions from a large number of unrelated true classes (digits, upper case and lower case alike), and upper-case Z and r similarly absorb many misclassifications. This pattern is typical of a linear one-vs-rest classifier that has not converged: the weight vectors for a few classes end up with large positive scores across much of the input space, so those classes “win” the arg-max on many unrelated inputs. A handful of genuinely visually similar pairs are still recovered correctly along the diagonal (e.g. C, J, L, N, T, Z, 8), showing the linear boundary works only for a minority of well-separated classes.

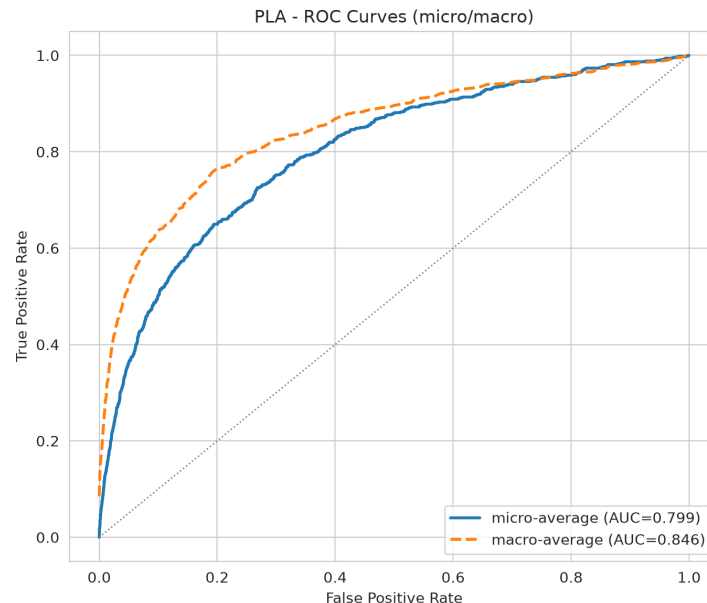


Figure 3: Micro- and macro-average ROC curves for the PLA (raw class scores used as ranking scores).

Inference: The PLA produces raw scores rather than calibrated probabilities, but ROC analysis only needs the ranking of scores, so the one-vs-rest curves remain valid. With 62 classes each binary problem has roughly 61 negatives per positive, so an AUC well above 0.5 can coexist with very low top-1 accuracy: here micro-AUC = 0.799 and macro-AUC = 0.846, both comfortably above chance despite the 13.6% top-1 accuracy. The macro average being noticeably higher than the micro average indicates that several individual classes are ranked quite well one-vs-rest even though the arg-max decision (which must pick a single best class among 62) fails much more often; the micro curve is dragged down by the "sink" classes (u , Z , r) that generate many false positives. Compare with the MLP curves below: the roughly 0.07–0.11 AUC gap shows how much better the MLP ranks the correct class.

10.2 Model B: Tuned MLP

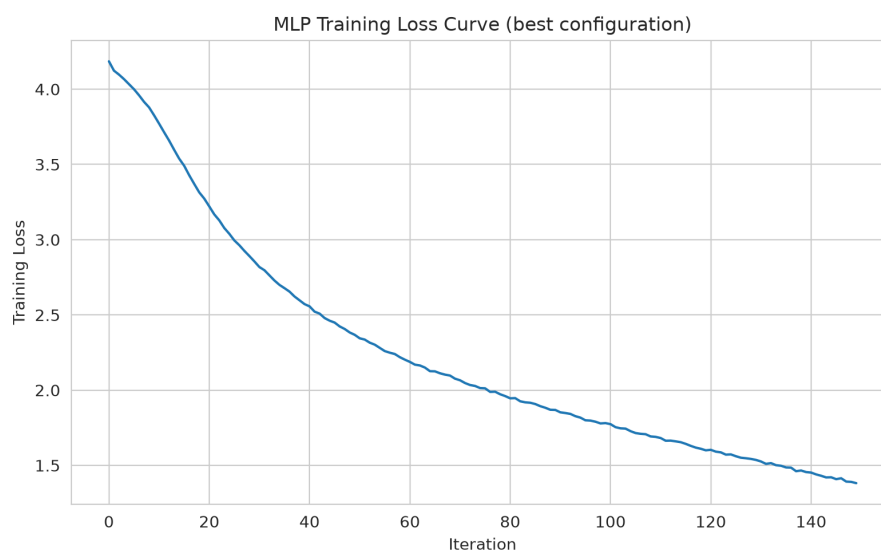


Figure 4: Training loss (cross-entropy) versus iteration for the best MLP configuration.

Inference: The training loss decreases steadily and smoothly with iterations, from about 4.2 (roughly $\ln 62 \approx 4.13$, the loss expected from a random 62-way softmax) down to about 1.4 at iteration 150, showing that backpropagation with SGD ($\eta = 0.001$) optimises the network consistently, unlike the oscillating PLA error. Crucially, the curve is still sloping downward at iteration 150 with no visible plateau, which matches the `ConvergenceWarning` scikit-learn raised (the 150-iteration cap was reached before the loss-tolerance criterion was met). This means the network was still improving when training stopped, and additional epochs (or a lower stopping tolerance) would likely push test accuracy higher still.

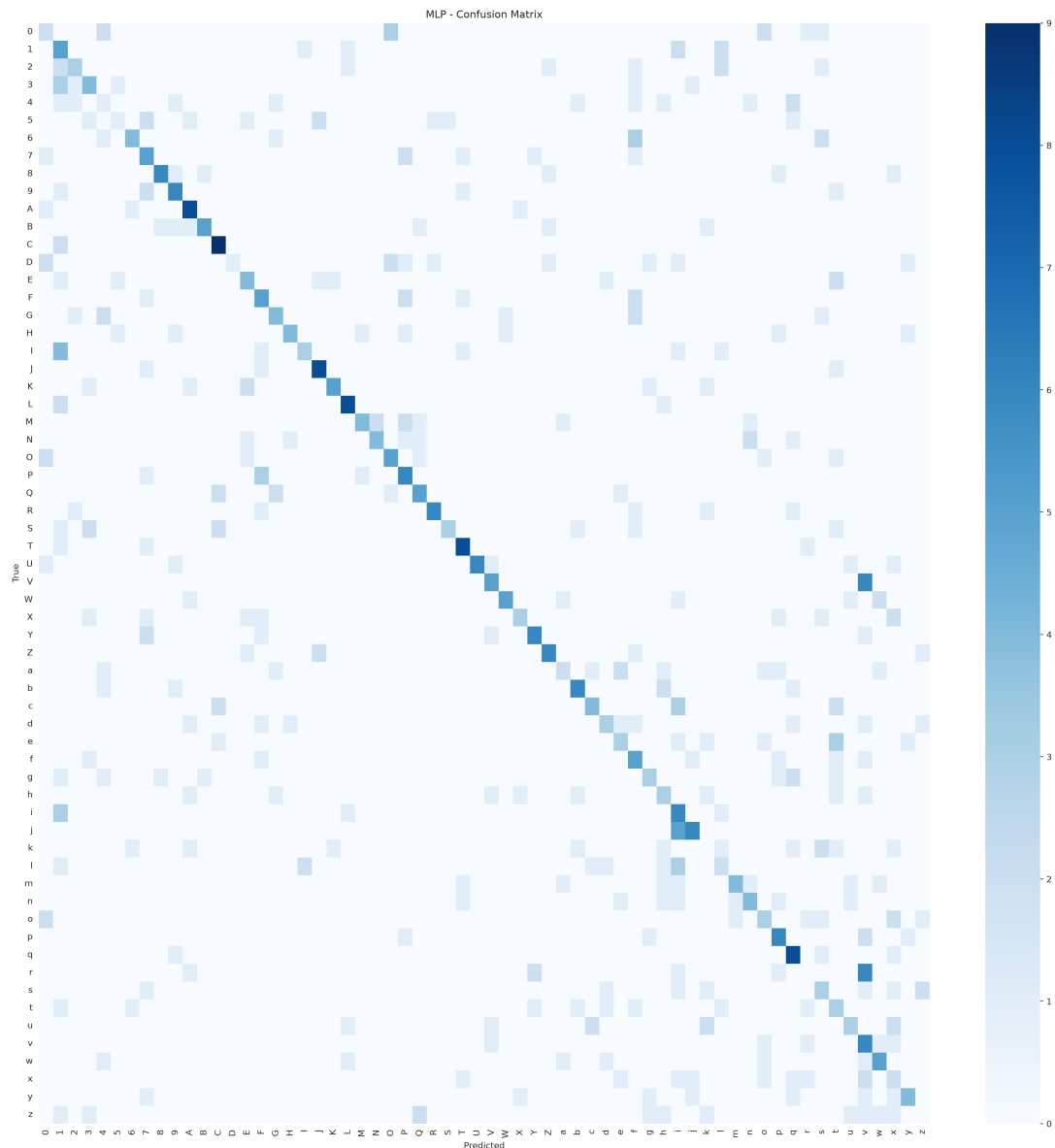


Figure 5: Confusion matrix of the tuned MLP on the test set (62 classes).

Inference: The diagonal of the MLP confusion matrix is clearly stronger than that of the PLA, matching the rise in test accuracy from 13.6% to 39.2%. There is no single dominant "sink" class as with the PLA; instead the remaining errors sit close to the diagonal and are concentrated among visually similar upper/lower-case pairs and digit/letter look-alikes, most visibly v/V, i/j, p/q, and w/W, together with scattered confusions among 0/O, 1/I/1, and C/c. This is exactly the kind of ambiguity that is very hard to resolve from raw 32×32 pixels alone, since these character pairs can differ only in size or a small stroke detail that a small amount of downsampling or handwriting variation can erase.

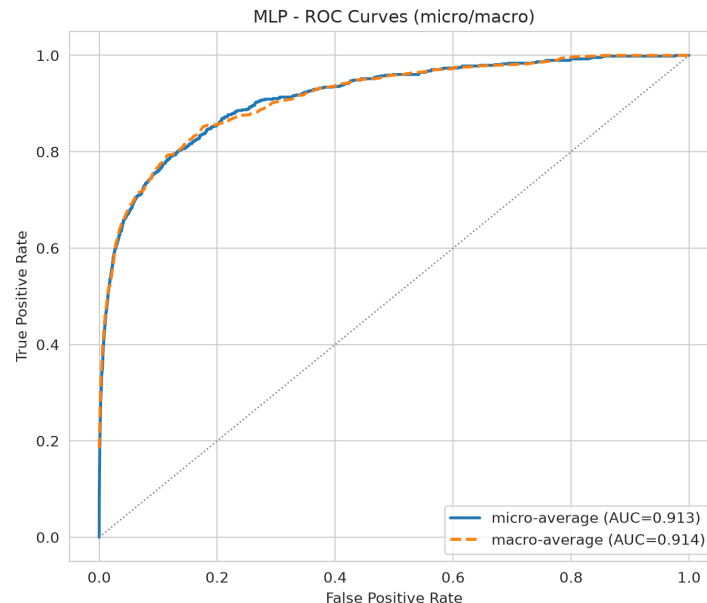


Figure 6: Micro- and macro-average ROC curves for the tuned MLP (softmax probabilities).

Inference: The MLP outputs softmax probabilities, giving well-behaved one-vs-rest ROC curves that are nearly indistinguishable between micro ($\text{AUC} = 0.913$) and macro ($\text{AUC} = 0.914$) averaging, indicating that performance is fairly consistent across classes rather than being carried by a few easy ones (unlike the PLA, where micro and macro AUC diverged by almost 0.05). The macro average weights every class equally and is therefore the fairer summary here because the classes are balanced. Both curves lie well above the diagonal and clearly above the PLA’s curves across the whole false-positive-rate range, confirming the MLP separates classes far more reliably even before the final arg-max decision is taken.

10.3 A/B comparison

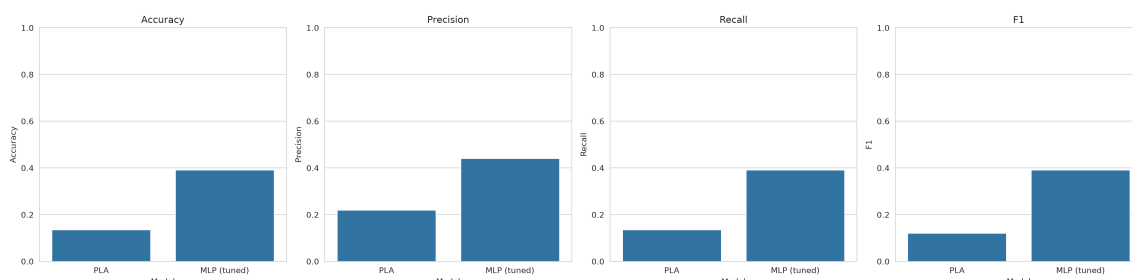


Figure 7: Test-set Accuracy, Precision, Recall and F1-score of PLA versus the tuned MLP.

Inference: The tuned MLP is better on all four metrics: accuracy rises from 0.136 to 0.391 (about 2.9 times higher, +25.5 points) and macro F1 from 0.121 to 0.391 (about 3.2 times). Both models are far above the 1.6% chance level, but an absolute accuracy of 39% shows that the task is hard for fully connected networks on raw 32×32 pixels

with only ~ 35 training images per class. The gain comes from the hidden layers' ability to model non-linear structure that a single linear layer cannot represent.

11 Discussion

11.1 Strengths and weaknesses: PLA vs MLP

	PLA	MLP
Strengths	Very simple and fast; easy to implement; interpretable per-class weight vectors	Non-linear boundaries; probabilistic outputs; trained with gradient-based backpropagation
Weaknesses	Linear only; does not converge on non-separable data; noisy validation error; no probabilities	Many hyperparameters; sensitive to optimizer and learning rate; needs more epochs and compute; risk of overfitting

11.2 Answers to observation questions

1. **Why does PLA underperform compared to MLP?** Handwritten characters in pixel space are not linearly separable: the same character varies greatly in stroke, slant and thickness, and many classes look alike (O/0/o, I/l/1). A single linear layer can only draw hyperplanes, and the perceptron rule never converges on such data, which is why its validation error keeps oscillating (0.75–0.96) and its confusion matrix shows predictions collapsing onto a few "sink" classes (u, Z, r). The MLP's hidden layers apply non-linear transformations and a cross-entropy loss with smooth gradients, so it can model far more complex class regions (39.2% vs 13.6% accuracy).
2. **Which hyperparameters had the most impact?** The *optimizer/learning-rate combination* had by far the largest effect: the same architecture ranged from about 1.5% (chance) to about 41% depending on solver and learning rate. Within SGD, lowering η from 0.01 to 0.001 improved validation accuracy in all six comparisons (by up to 22 points, e.g. (128) ReLU: 0.015 $\rightarrow$ 0.379). The choice of activation (ReLU vs tanh) changed accuracy by only about 1 point, and network depth/width changed it by about 1–3 points.
3. **Did optimizer choice (SGD vs Adam) affect convergence?** Yes, strongly—but in the opposite direction from common expectation. With $\eta \in \{0.001, 0.01\}$ Adam runs stayed at or near chance level (1.5–9.7% validation accuracy), i.e. the networks did not learn a useful classifier, while SGD with $\eta = 0.001$ reached 38–41%. Likely reasons are that Adam's per-parameter adaptive steps at these learning rates are too aggressive for un-standardised, highly redundant 1024-dimensional pixel inputs with few samples per class, leading to unstable updates or saturated/dead units. This was not verified experimentally. Smaller Adam learning rates (e.g. 10^{-4}), input standardisation, or early stopping should be tried. SGD, in contrast, was slow: it hit the 150-iteration limit without meeting the convergence tolerance (the loss curve in Figure 4 is still decreasing at iteration 150 with no plateau).

4. **Did adding more hidden layers always improve results?** No. Using SGD with $\eta = 0.001$, going from (128) to (128, 64) to (256, 128) raised validation accuracy only slightly (ReLU: $0.379 \rightarrow 0.403 \rightarrow 0.407$; tanh: $0.388 \rightarrow 0.394 \rightarrow 0.414$). With 546 validation images the standard error of an accuracy near 0.4 is about $\sqrt{0.4 \cdot 0.6/546} \approx 2.1$ points, so these differences are of the same order as the noise. Extra depth also did nothing for Adam, which failed at every depth. More layers add capacity but also make optimisation harder and increase overfitting risk when data are scarce; here the limiting factors are data quantity and the optimiser, not depth.
5. **Did the MLP show overfitting? How could it be mitigated?** The best model scored 0.414 on validation and 0.392 on test, a gap of about 2 points that is within sampling noise (and partly caused by selecting the best of 24 configurations on the validation set), so there is no clear overfitting in these results. However, training accuracy was not recorded, so the train-validation gap could not be measured, and the largest network has about 3×10^5 parameters for only 2,182 training images, so overfitting is a real risk if training continues (especially since the loss curve had not plateaued at 150 iterations). Mitigation: a larger L_2 penalty (α), early stopping on the validation set, smaller networks, dropout (in a deep-learning framework), data augmentation (small shifts, rotations, elastic distortions), more training data, and dimensionality reduction (e.g. PCA) or convolutional layers to exploit image structure.

11.3 Limitations and possible improvements

- Test accuracy of the MLP is only 39%; fully connected networks on raw pixels are not well suited to this task. A CNN with data augmentation would be expected to do considerably better.
- SGD runs did not reach convergence within 150 iterations; more epochs, a learning-rate schedule or early stopping would allow a fairer comparison.
- Standardising pixel features (zero mean, unit variance) and tuning Adam with smaller learning rates could make Adam competitive.
- Results come from a single random split and seed; repeated runs or k -fold cross-validation would give confidence intervals and a more reliable ranking of configurations.
- The grid was limited to batch size 32 and three architectures; larger searches (random search, batch size, α) may find better settings.

12 Conclusion

A multiclass single-layer perceptron (implemented from scratch) and a tuned Multilayer Perceptron were compared on the 62-class English Handwritten Characters dataset. The PLA reached only 13.6% test accuracy (macro F1 0.121, ROC-AUC 0.799/0.846 micro/macro) and its validation error oscillated because the data are not linearly separable. After a 24-configuration grid search, the best MLP (hidden layers (256, 128), tanh, SGD, $\eta = 0.001$, batch size 32) achieved 39.2% test accuracy (macro F1 0.391, ROC-AUC 0.913/0.914 micro/macro), about 2.9 times better than the PLA on accuracy and a clear

improvement in ranking quality as well. Hyperparameter tuning was decisive: the choice of optimizer and learning rate moved accuracy from chance level to about 40%, while activation function and depth had a comparatively small effect. The absolute accuracy remains modest, so future work should include feature standardisation, longer training with regularisation, data augmentation, and convolutional architectures.

References

- [1] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958.
- [2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986.
- [3] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.
- [4] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [6] D. Dave, “English Handwritten Characters Dataset,” Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/dhruvildave/english-handwritten-characters-dataset>